

■ PROJECT REPORT — FLIGHT BOOKING APPLICATION

1.Introduction:

- **Project Title:** FlightFinder: Navigating Your Air Travel Options
- **Team ID:** LTVIP2026TMIDS65742
- **Team members:**
 - Kanuru Sai Vinay
 - Katragadda Uday Kiran
 - Kanna Nithin Ashok Kumar
 - Kornepati Purushotham

2.Project Overview:

- **Purpose:**

The purpose of the Flight Booking Application is to provide a simple and user-friendly web platform that allows users to search and book flights efficiently. The system reduces manual effort by digitizing the booking process and helps users manage their bookings easily. It also provides an admin interface to manage flight information within the system.
- **Features:**
 - User Registration and Login
 - Secure Authentication (JWT-based)
 - Flight Search using source, destination, and date
 - View Flight Details (timing, price, airline info)
 - Flight Booking with Passenger Details
 - Booking Confirmation generation
 - View Booking History
 - Admin Dashboard for Flight Management
 - Add, Update, and Delete Flights (Admin)

3.Architecture:

- **Frontend:**

The frontend is developed using Next.js, a React-based framework that supports component-based architecture and efficient client-side rendering. It provides interactive user interfaces for authentication, flight search, booking, and admin operations. The frontend communicates with backend services through RESTful APIs using HTTP requests and dynamically updates the UI based on server responses. This separation ensures maintainability and scalability of the presentation layer.

- **Backend:**

The backend is implemented using Node.js with Express.js, following a modular REST API architecture. It handles core business logic including user authentication,

flight search processing, booking management, and admin functionalities. Middleware is used for request validation, routing, and authentication using JWT tokens. The backend acts as an intermediary between the frontend and database, ensuring secure data handling and efficient processing of client requests.

- **Database:**

The system uses MongoDB, a NoSQL document-oriented database, to store and manage application data. Collections are used to maintain structured data such as users, flights, and bookings. MongoDB enables flexible schema design, fast query performance, and easy scalability, making it suitable for handling dynamic flight and booking data. Database operations are performed through backend APIs to ensure data consistency and security.

4.Setup Instructions:

- **Prerequisites:**

Before running the application, ensure the following software and tools are installed:

- Node.js (v16 or above)
- npm (Node Package Manager)
- MongoDB (Local installation or MongoDB Atlas)
- Git (for cloning the repository)
- Code Editor (Visual Studio Code recommended)
- Modern Web Browser (Chrome / Edge)

- **Installation:**

Step 1: Clone the Repository

```
git clone https://github.com/SaivinayK007/FlightFinder.git
```

Step 2: Install Frontend Dependencies

```
cd client
```

```
npm install
```

Step 3: Install Backend Dependencies

```
cd server
```

```
npm install
```

Step 4: Configure Environment Variables

Create a `.env` file inside the server folder and add:

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_secret_key
```

Step 5: Start Backend Server

```
npm start
```

Step 6: Start Frontend Server

```
npm run dev
```

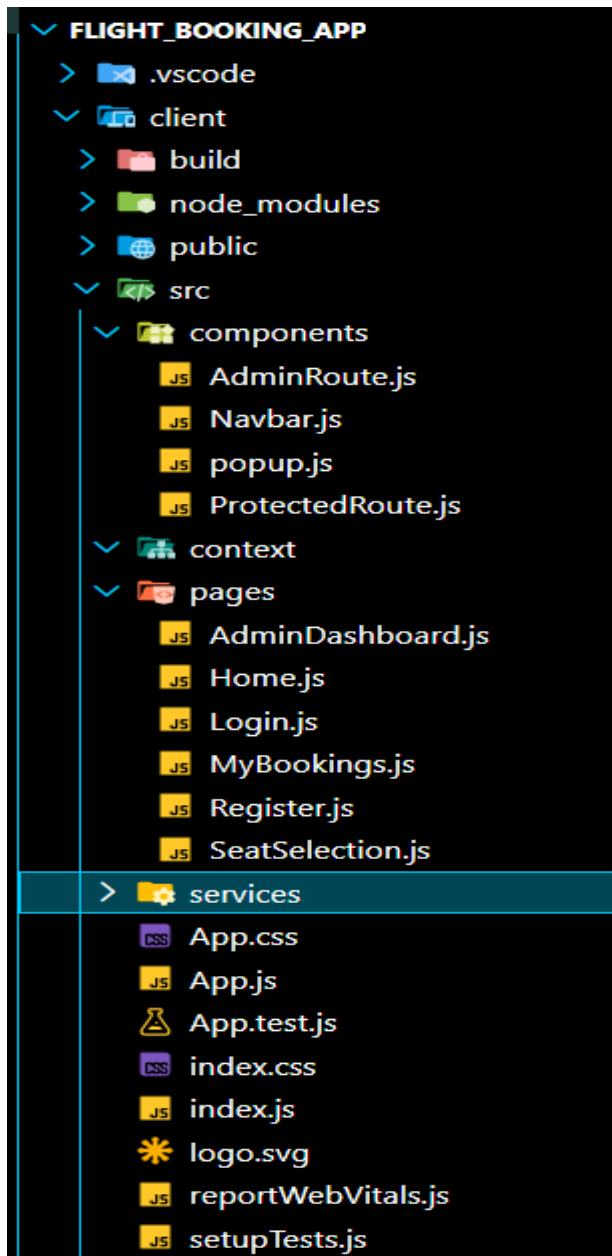
Step 7:Run Application

Use 2 terminals to start each client and server module separately or use the command in root terminal to start the application

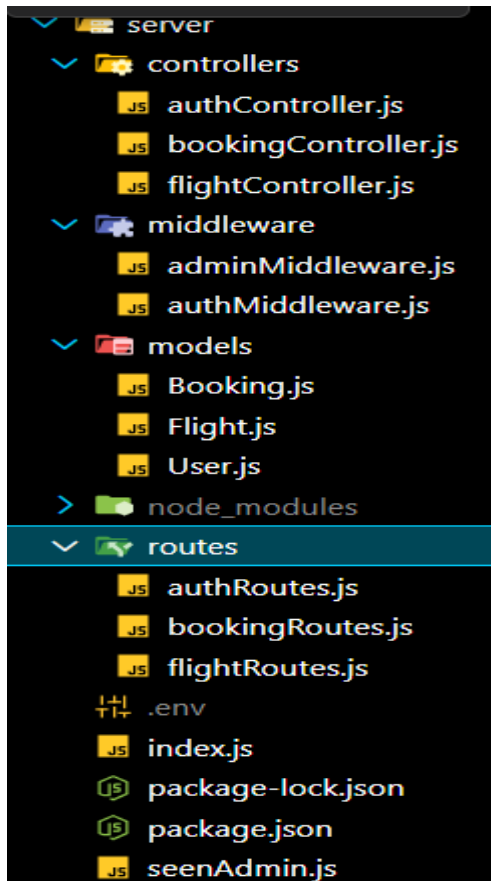
`npm run dev`

5.Folder Structure:

- **Client:**The client folder contains the frontend code developed using React.js. It manages user interface, routing, and interaction with backend APIs.



- **Server:**
The server folder contains Backend logic built using [Node.js](#) and [Express.js](#)



6. Running the Application:

- **Frontend:**
 - Navigate to client folder
 - Cd client
 - Install dependencies
 - npm install
 - Start Frontend Server
 - npm start
 - This backend runs on <http://localhost:3000>
- **Backend:**
 - Navigate to Server Folder
 - cd server
 - Install Dependencies
 - npm install
 - Start Backend server
 - npm start
 - This runs on <http://localhost:5000>

7.API Documentation:

- Authentication APIs:

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	User login

- Flight APIs:

Method	Endpoint	Description
GET	/api/flights	Get all flights
GET	/api/flights/:id	Get flight details
POST	/api/flights	Add flight (Admin)
PUT	/api/flights/:id	Update flight (Admin)
DELETE	/api/flights/:id	Delete flight (Admin)

- Booking APIs:

Method	Endpoint	Description
POST	/api/bookings	Create booking
GET	/api/bookings/user/:id	Get user bookings
GET	/api/bookings	Get all bookings (Admin)

8.Authentication:

- The application uses JWT-based authentication to secure user access and protect routes. Users can register using email and password, and credentials are validated during login. Passwords are encrypted using hashing before storing in the database to ensure security.
- After successful login, a JSON Web Token (JWT) is generated by the backend and used to authenticate API requests. Authentication

middleware verifies the token before allowing access to protected routes such as booking history and admin functionalities. Role-based access control is used to differentiate normal users and administrators.

9.User Interface:

The user interface is developed using React.js and designed to provide a simple and intuitive user experience.

Main UI Screens:

- Login Page
- Registration Page
- Home / Flight Search Page
- Flight Selection / Seat Selection Page
- My Bookings Page
- Admin Dashboard

UI Characteristics:

- Clean and responsive layout
- Easy navigation using React Router
- Form validation for user input
- Dynamic rendering of flight and booking data
- Component-based design for reusability

The interface focuses on reducing complexity and enabling users to complete booking operations with minimal steps.

10.Testing:

Testing was performed to ensure correct functionality of all implemented modules.

Types of Testing Performed:

- Functional Testing
- User Acceptance Testing (UAT)
- Validation Testing

Modules Tested:

- User Registration & Login

- Flight Search
- Flight Booking
- Booking History
- Admin Flight Management

Testing Results:

- All major functionalities executed successfully.
Input validation worked correctly.
- API communication between frontend and backend was verified.
- Minor UI and validation issues were identified and fixed during testing.

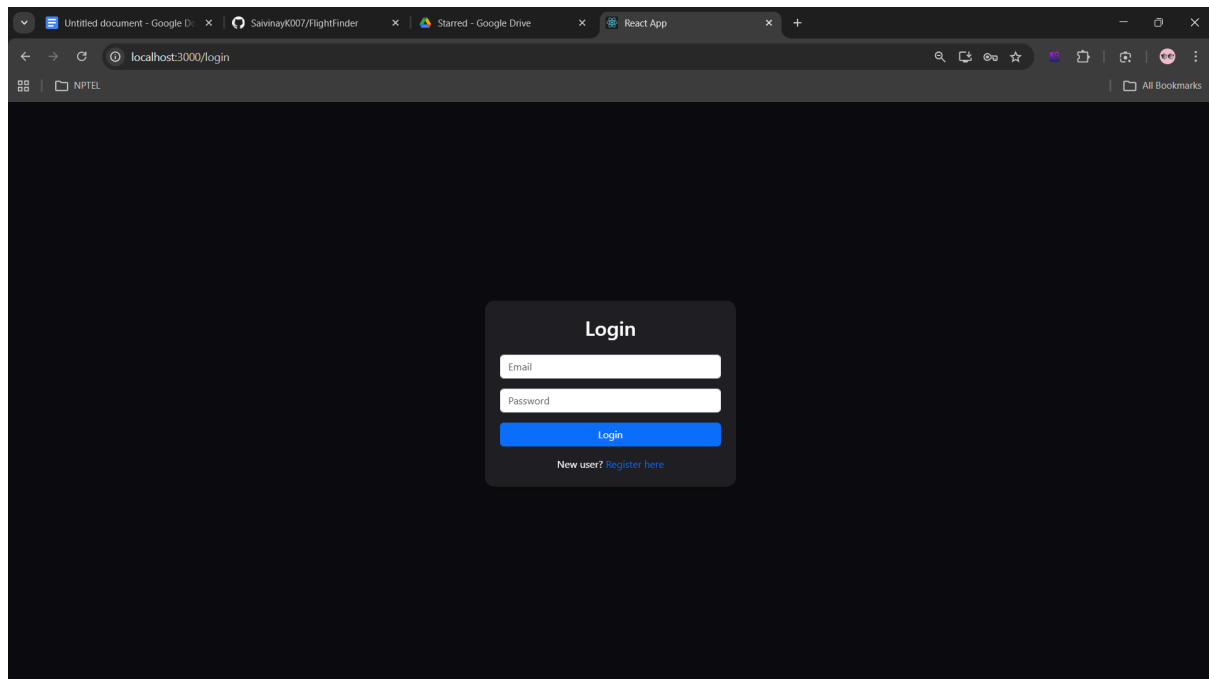
The application met the expected requirements and produced stable results during testing.

11.Screenshots or Demo:

The Application can be started by the using “npm run dev” in root folder

The following is the pages displayed

[Login.js](#)



We have 2 types of users :

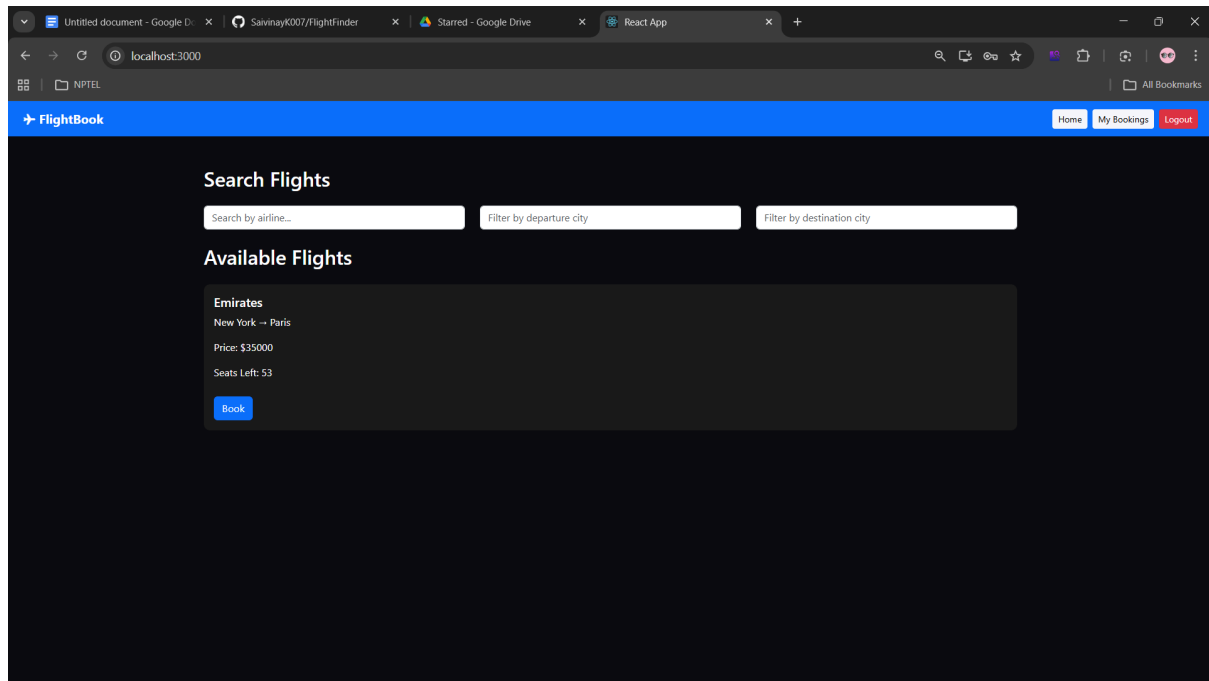
- Admin
- User

If logged in as user the Home page is displayed as :

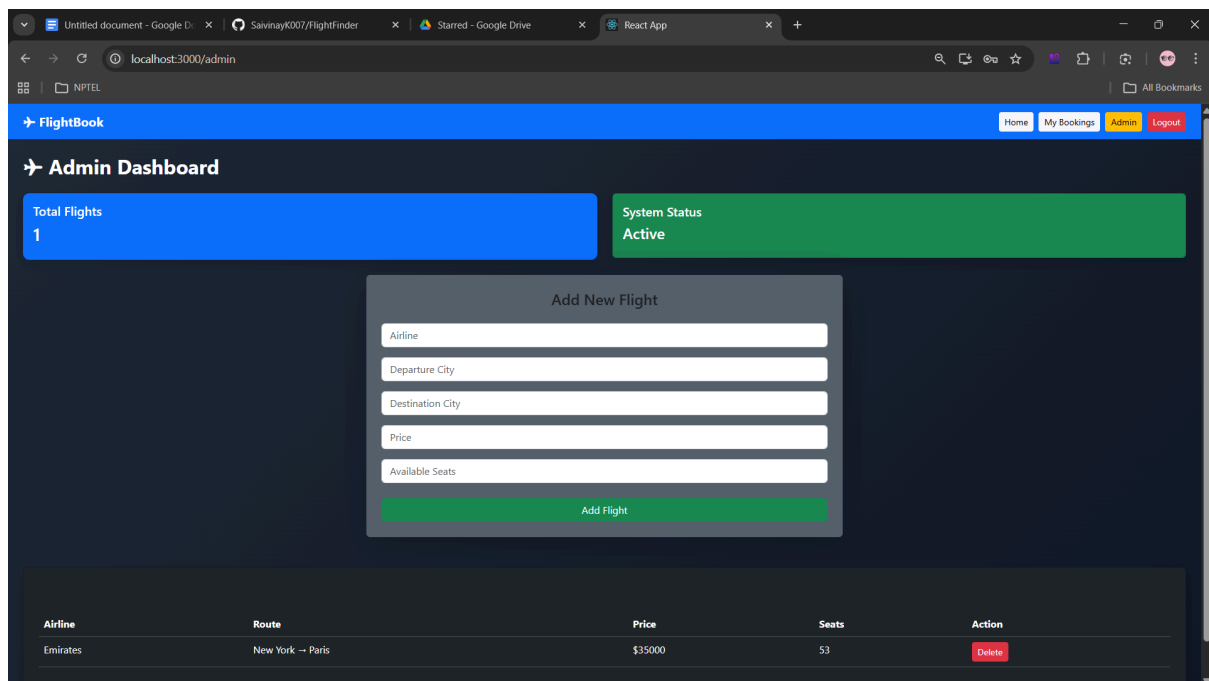
Existing User : [John@gmail.com](#)

Password:123456

Home.js



If logged in as admin then the following page will appear
AdminDashboard.js:



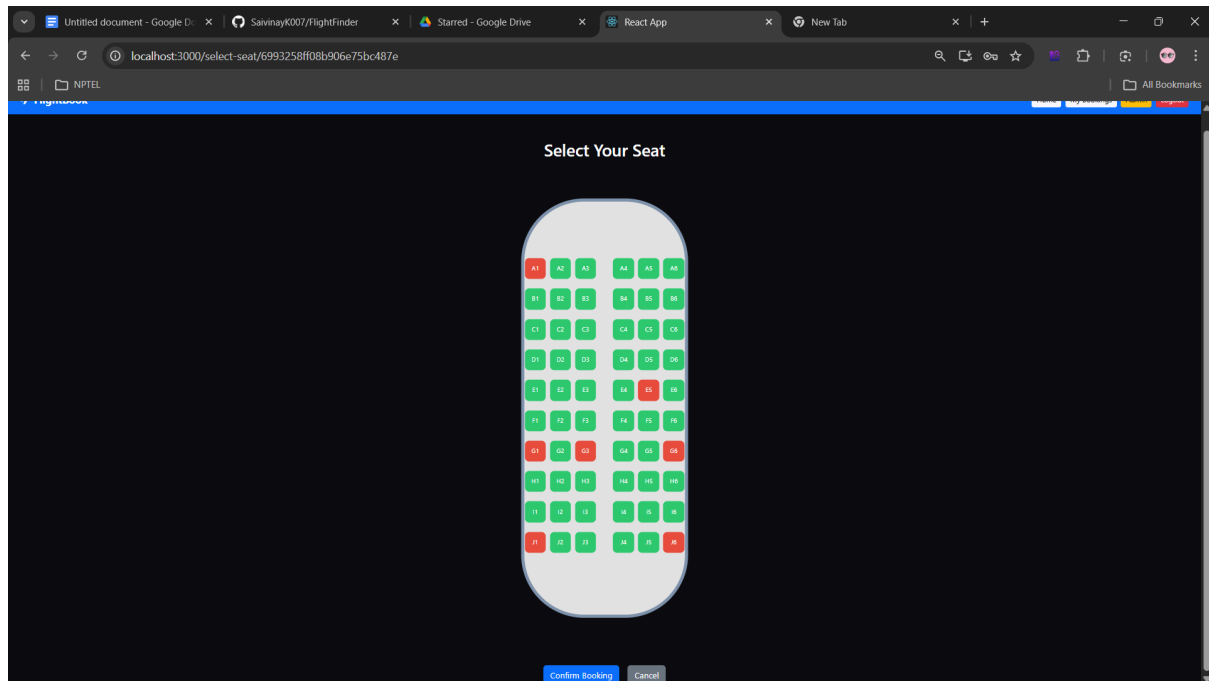
Details for admin were:

mail: admin@gmail.com

password: admin123

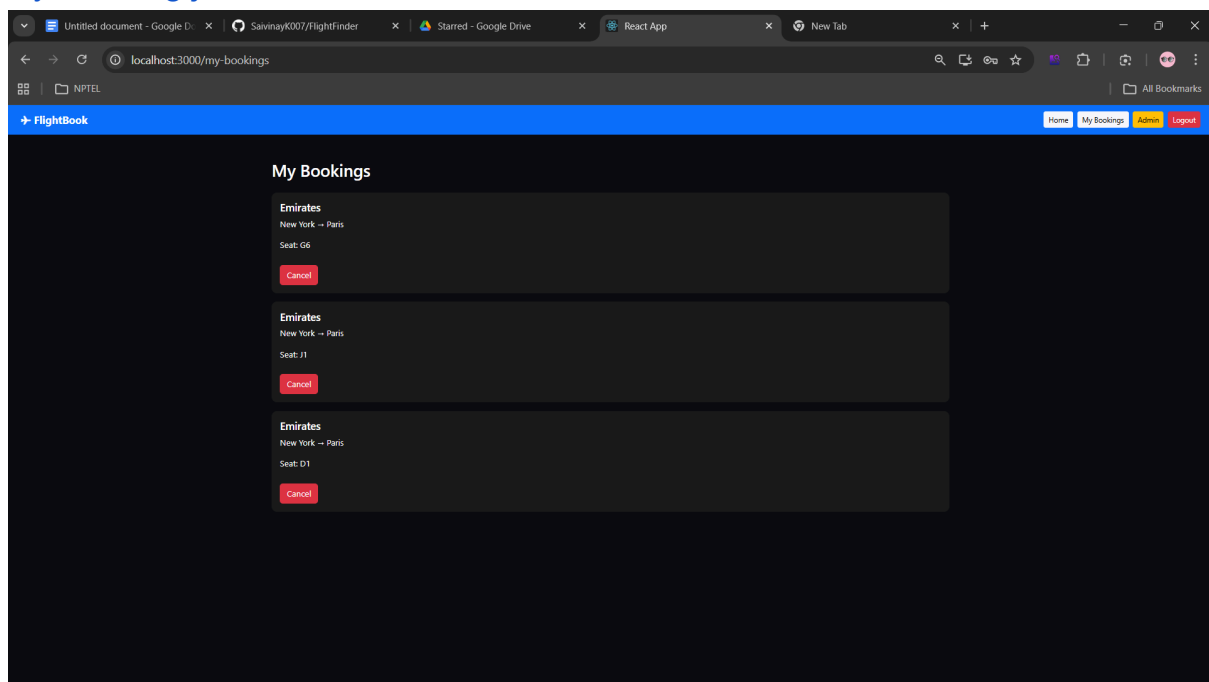
The admin can add, update and delete flights

In order to book tickets,you will get [SeatSelection.js](#)



Where available seats are in green and already booked seats are red,the seat you selected as Blue and confirm the ticket which will be shown in my bookings section as follows

[my-booking.js](#)



Here is the list of booking done by logged in user you can cancel the booking.

Here is the details of MongoDB compass

Users:

MongoDB Compass - localhost:27017/FlightDb.users

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (4)

Search connections

✱ vltts

cluster0.vjwsec.mongodb.net

localhost:27017

FlightDb

bookings

flight

flights

users

admin

booktracker

cars

citizenai

config

local

rentacar

rentacar

localhost:27017 > FlightDb > users

Documents (4) Aggregations Schema Indexes (2) Validation

Type a query: (field: 'value') or [Generate query](#)

Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

25 1 - 4 of 4

```
{
  "_id": ObjectId("6992d8571d4d0208fe9aa0ba"),
  "name": "John",
  "email": "john@gmail.com",
  "password": "52b51853bW1I.fBa.g1R2mRwb9PG.c2dgdFULZv9DYp5073eYIsbQDuB5nne",
  "role": "user",
  "createdAt": 2026-02-16T08:07:51.316+00:00,
  "updatedAt": 2026-02-16T08:07:51.316+00:00,
  "__v": 0
}
```

```
{
  "_id": ObjectId("6992e8a95dea00846de81475"),
  "name": "Admin",
  "email": "admin@gmail.com",
  "password": "52b51853bW1I.fBa.g1R2mRwb9PG.c2dgdFULZv9DYp5073eYIsbQDuB5nne",
  "role": "admin",
  "createdAt": 2026-02-16T09:51:28.218+00:00,
  "updatedAt": 2026-02-16T09:51:28.218+00:00,
  "__v": 0
}
```

```
{
  "_id": ObjectId("699343aac52e2b439d6b3f65"),
  "name": "Salvino",
  "email": "kylecrane725@gmail.com",
  "password": "52b51853bW1I.fBa.g1R2mRwb9PG.c2dgdFULZv9DYp5073eYIsbQDuB5nne",
  "role": "user",
  "createdAt": 2026-02-16T16:19:54.436+00:00,
  "updatedAt": 2026-02-16T16:19:54.436+00:00,
  "__v": 0
}
```

Bookings:

MongoDB Compass - localhost:27017/FlightDb.bookings

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (4)

Search connections

✱ vltts

cluster0.vjwsec.mongodb.net

localhost:27017

FlightDb

bookings

flight

flights

users

admin

booktracker

cars

citizenai

config

local

rentacar

rentacar

localhost:27017 > FlightDb > bookings

Documents (8) Aggregations Schema Indexes (2) Validation

Type a query: (field: 'value') or [Generate query](#)

Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

25 1 - 8 of 8

```
{
  "_id": ObjectId("69933889f8bb986e75bc4985"),
  "user": ObjectId("6992d8571d4d0208fe9aa0ba"),
  "flight": ObjectId("6993258ff8bb986e75bc487e"),
  "seatNumber": "A1",
  "status": "Booked",
  "createdAt": 2026-02-16T15:30:17.764+00:00,
  "updatedAt": 2026-02-16T15:30:17.764+00:00,
  "__v": 0
}
```

```
{
  "_id": ObjectId("69945ad4b56e5363a9fa8ca9"),
  "user": ObjectId("6992d8571d4d0208fe9aa0ba"),
  "flight": ObjectId("6993258ff8bb986e75bc487e"),
  "seatNumber": "36",
  "status": "Booked",
  "createdAt": 2026-02-17T12:11:00.389+00:00,
  "updatedAt": 2026-02-17T12:11:00.389+00:00,
  "__v": 0
}
```

```
{
  "_id": ObjectId("69947ea6b56e5363a9fa8f2f"),
  "user": ObjectId("6992e8a95dea00846de81475"),
  "flight": ObjectId("6993258ff8bb986e75bc487e"),
  "seatNumber": "60",
  "status": "Booked",
  "createdAt": 2026-02-17T14:43:50.778+00:00,
  "updatedAt": 2026-02-17T14:43:50.778+00:00,
  "__v": 0
}
```

Flights:

MongoDB Compass - localhost:27017/FlightDb.flights

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (4)

Search connections

✱ vltts

cluster0.vjwsec.mongodb.net

localhost:27017

FlightDb

bookings

flight

flights

users

admin

booktracker

cars

citizenai

config

local

rentacar

rentacar

localhost:27017 > FlightDb > flights

Documents (1) Aggregations Schema Indexes (1) Validation

Type a query: (field: 'value') or [Generate query](#)

Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

25 1 - 1 of 1

```
{
  "_id": ObjectId("6993258ff8bb986e75bc487e"),
  "airline": "Emirates",
  "departureCity": "New York",
  "destinationCity": "Paris",
  "price": 35000,
  "availableSeats": 50,
  "createdAt": 2026-02-16T14:11:27.676+00:00,
  "updatedAt": 2026-02-20T08:28:30.580+00:00,
  "__v": 0
}
```

12. Known Issues

Despite successful implementation, the flight booking application has several limitations:

1. **Limited Payment Integration:**

The current system may not include complete payment gateway integration, restricting real-world transaction handling.

2. **Static Flight Data:**

Flight details may rely on predefined or manually added data instead of real-time airline APIs.

3. **Scalability Constraints:**

The application performance may decrease with a large number of concurrent users due to limited backend optimization.

4. **Basic Security Measures:**

Although authentication is implemented, advanced security features such as rate limiting, OTP verification, and encryption enhancements are not fully integrated.

5. **Limited Admin Controls:**

Administrative functionalities like analytics, flight management dashboards, and dynamic pricing are minimal.

13. Future Enhancements

The following enhancements can improve the functionality and usability of the flight booking system:

1. **Payment Gateway Integration:**

Integrating secure payment systems such as Razorpay, Stripe, or PayPal for real-time booking confirmation.

2. **Real-Time Flight APIs:**

Connecting with airline or travel APIs to fetch live flight schedules, pricing, and seat availability.

3. **Advanced Search and Filters:**

Adding filters based on price, airline, duration, stops, and ratings to improve user experience.

4. Admin Dashboard:

Implementing a complete admin panel for managing flights, bookings, users, and analytics.

5. Notification System:

Adding email/SMS notifications for booking confirmation, cancellation, and flight updates.

6. Performance Optimization:

Using caching and backend optimization techniques to handle higher traffic efficiently.

Source Code : <https://github.com/SaivinayK007/FlightFinder.git>

Project Demo : 📺 FSD project Demo Link (FlightFinder).mp4